Industry Protocols: Lab 3 (AMBA-AXI)

Submitted by: kashish shaikh

Master:

```
module AXI_master #(
parameter ADDRESS = 32,
parameter DATA_WIDTH = 32
)(
input wire ACLK,
input wire ARESETN,
input wire START_READ,
input wire START_WRITE,
input wire [ADDRESS-1:0] address,
input wire [DATA_WIDTH-1:0] W_data,
input wire ARREADY,
input wire [DATA_WIDTH-1:0] RDATA,
input wire [1:0] RRESP,
input wire RVALID,
input wire AWREADY,
input wire WREADY,
input wire [1:0] BRESP,
input wire BVALID,
output reg [ADDRESS-1:0] ARADDR,
output reg ARVALID,
output reg RREADY,
output reg [ADDRESS-1:0] AWADDR,
output reg AWVALID,
output reg [DATA_WIDTH-1:0] WDATA,
output reg [3:0] WSTRB,
output reg WVALID,
output reg BREADY
);
localparam IDLE = 3'b000;
localparam RADDR_CHANNEL = 3'b001;
localparam RDATA_CHANNEL = 3'b010;
localparam WRITE_CHANNEL = 3'b011;
localparam WRESP_CHANNEL = 3'b100;
reg [2:0] state;
always @(posedge ACLK or negedge ARESETN) begin
if (!ARESETN) begin
state <= IDLE;
ARADDR <= 0;
ARVALID <= 0;
RREADY <= 0;
AWADDR <= 0;
AWVALID <= 0;
WDATA <= 0;
WSTRB <= 4'b0000;
WVALID <= 0;
BREADY <= 0;
```

```
46  BREADY <= 0;
47  end
48  else begin
49  case (state)
50  IDLE: begin
51  ARVALID <= 0;
52  RREADY <= 0;
53  AWVALID <= 0;
54  WVALID <= 0;
55  BREADY <= 0;
56  if (START_READ) begin
57  ARADDR <= address;
58  ARVALID <= 1;
59  state <= RADDR_CHANNEL;
60  end
61  else if (START_WRITE) begin
62  AWADDR <= address;
63  WDATA <= W_data;
64  AWVALID <= 1;
65  WVALID <= 1;
66  WSTRB <= 4'b1111;
67  state <= WRITE_CHANNEL;
68  end
69  end
70  RADDR_CHANNEL: begin
71  if (ARVALID && ARREADY) begin
72  ARVALID <= 0;
73  RREADY <= 1;
74  state <= RDATA_CHANNEL;
75  end
76  end
77  RDATA_CHANNEL: begin
78  if (RVALID && RREADY) begin
79  RREADY <= 0;
80  state <= IDLE;
81  end
82  end
83  WRITE_CHANNEL: begin
84  if (AWVALID && AWREADY)
85  AWVALID <= 0;
86  if (WVALID && WREADY)
87  WVALID <= 0;
88  if ((!AWVALID || AWREADY) && (!WVALID || WREADY)) begin
89  BREADY <= 1;
90  state <= WRESP_CHANNEL;
91  end
```

```
82      end
83      WRITE_CHANNEL: begin
84      if (AWVALID && AWREADY)
85      AWVALID <= 0;
86      if (WVALID && WREADY)
87      WVALID <= 0;
88      if ((!AWVALID || AWREADY) && (!WVALID || WREADY)) begin
89      BREADY <= 1;
90      state <= WRESP_CHANNEL;
91      end
92      end
93      WRESP_CHANNEL: begin
94      if (BVALID && BREADY) begin
95      BREADY <= 0;
96      state <= IDLE;
97      end
98      end
99      default: begin
100     state <= IDLE;
101     end
102     endcase
103     end
104     end
105     endmodule
```

Slave:

```
Ln#
 1  module AXI_slave #(
 2  parameter ADDRESS = 32,
 3  parameter DATA_WIDTH = 32
 4  )(
 5  input wire ACLK,
 6  input wire ARESETN,
 7  input wire [ADDRESS-1:0] ARADDR,
 8  input wire ARVALID,
 9  input wire RREADY,
10  input wire [ADDRESS-1:0] AWADDR,
11  input wire AWVALID,
12  input wire [DATA_WIDTH-1:0] WDATA,
13  input wire [3:0] WSTRB,
14  input wire WVALID,
15  input wire BREADY,
16  output reg ARREADY,
17  output reg [DATA_WIDTH-1:0] RDATA,
18  output reg [1:0] RRESP,
19  output reg RVALID,
20  output reg AWREADY,
21  output reg WREADY,
22  output reg [1:0] BRESP,
23  output reg BVALID
24  );
25  reg [DATA_WIDTH-1:0] REG0;
26  reg [DATA_WIDTH-1:0] REG1;
27  reg [DATA_WIDTH-1:0] REG2;
28  reg [DATA_WIDTH-1:0] REG3;
29  always @(posedge ACLK or negedge ARESETN) begin
30  if (!ARESETN) begin
31  ARREADY <= 0;
32  RDATA <= 0;
33  RRESP <= 0;
34  RVALID <= 0;
35  AWREADY <= 0;
36  WREADY <= 0;
37  BRESP <= 0;
38  BVALID <= 0;
39  REG0 <= 0;
40  REG1 <= 0;
41  REG2 <= 0;
42  REG3 <= 0;
43  end
44  else begin
45  ARREADY <= 1;
46  AWREADY <= 1;
```

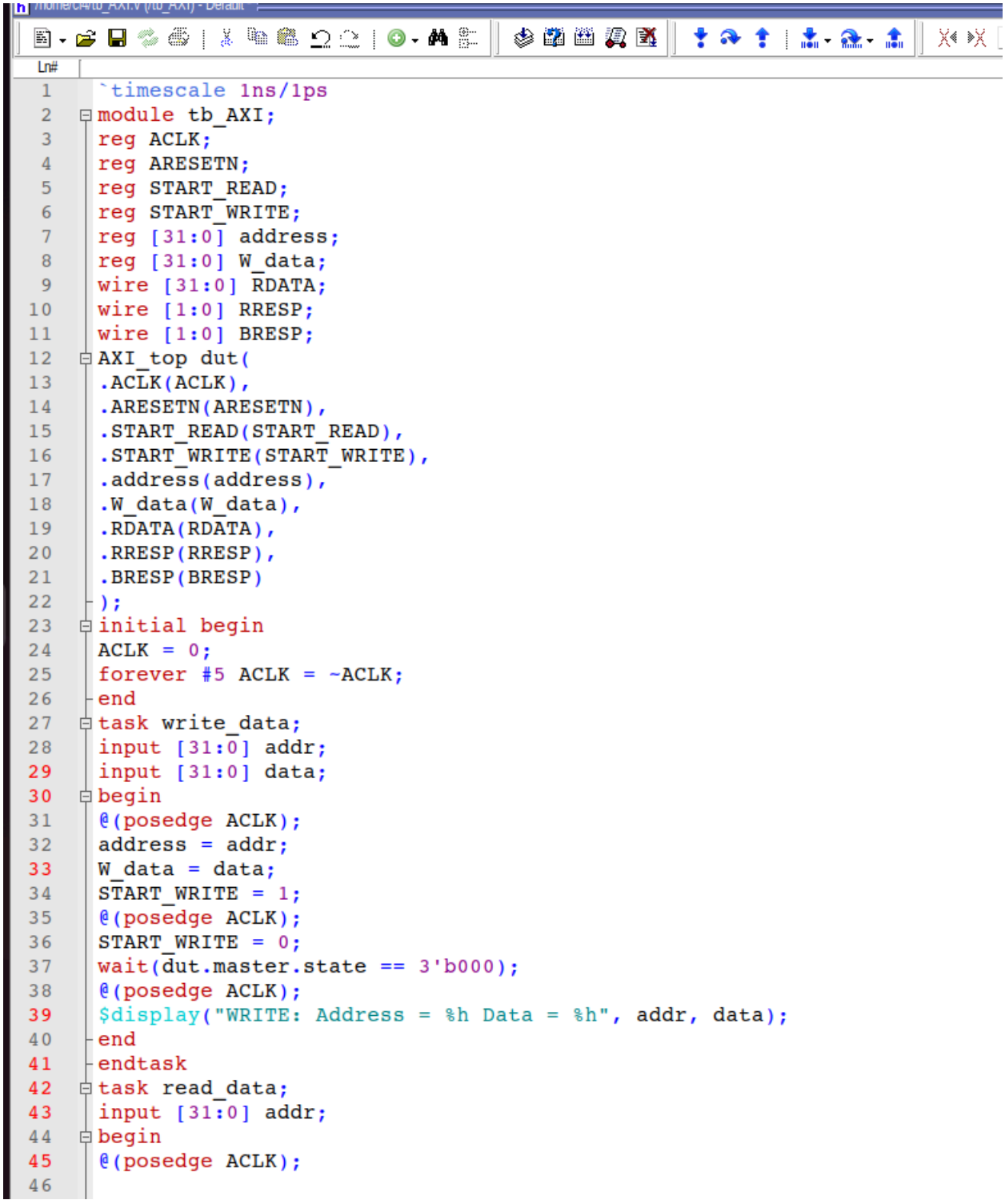
```
41  REG2 <= 0;
42  REG3 <= 0;
43  end
44  else begin
45  ARREADY <= 1;
46  AWREADY <= 1;
47  WREADY <= 1;
48  if (ARVALID && ARREADY) begin
49  case (ARADDR)
50  32'h00000000: RDATA <= REG0;
51  32'h00000004: RDATA <= REG1;
52  32'h00000008: RDATA <= REG2;
53  32'h0000000C: RDATA <= REG3;
54  default: RDATA <= 32'h00000000;
55  endcase
56  RRESP <= 2'b00;
57  RVALID <= 1;
58  end
59  if (RVALID && RREADY) begin
60  RVALID <= 0;
61  end
62  if (AWVALID && AWREADY && WVALID && WREADY) begin
63  case (AWADDR)
64  32'h00000000: begin
65  if (WSTRB[0]) REG0[7:0] <= WDATA[7:0];
66  if (WSTRB[1]) REG0[15:8] <= WDATA[15:8];
67  if (WSTRB[2]) REG0[23:16] <= WDATA[23:16];
68  if (WSTRB[3]) REG0[31:24] <= WDATA[31:24];
69  end
70  32'h00000004: begin
71  if (WSTRB[0]) REG1[7:0] <= WDATA[7:0];
72  if (WSTRB[1]) REG1[15:8] <= WDATA[15:8];
73  if (WSTRB[2]) REG1[23:16] <= WDATA[23:16];
74  if (WSTRB[3]) REG1[31:24] <= WDATA[31:24];
75  end
76  32'h00000008: begin
77  if (WSTRB[0]) REG2[7:0] <= WDATA[7:0];
78  if (WSTRB[1]) REG2[15:8] <= WDATA[15:8];
79  if (WSTRB[2]) REG2[23:16] <= WDATA[23:16];
80  if (WSTRB[3]) REG2[31:24] <= WDATA[31:24];
81  end
82  32'h0000000C: begin
83  if (WSTRB[0]) REG3[7:0] <= WDATA[7:0];
84  if (WSTRB[1]) REG3[15:8] <= WDATA[15:8];
85  if (WSTRB[2]) REG3[23:16] <= WDATA[23:16];
86  if (WSTRB[3]) REG3[31:24] <= WDATA[31:24];
87  end
88  endcase
89  BRESP <= 2'b00;
90  BVALID <= 1;
91  end
92  if (BVALID && BREADY) begin
93  BVALID <= 0;
94  end
95  end
96  end
97  endmodule
98
99
```

Top:

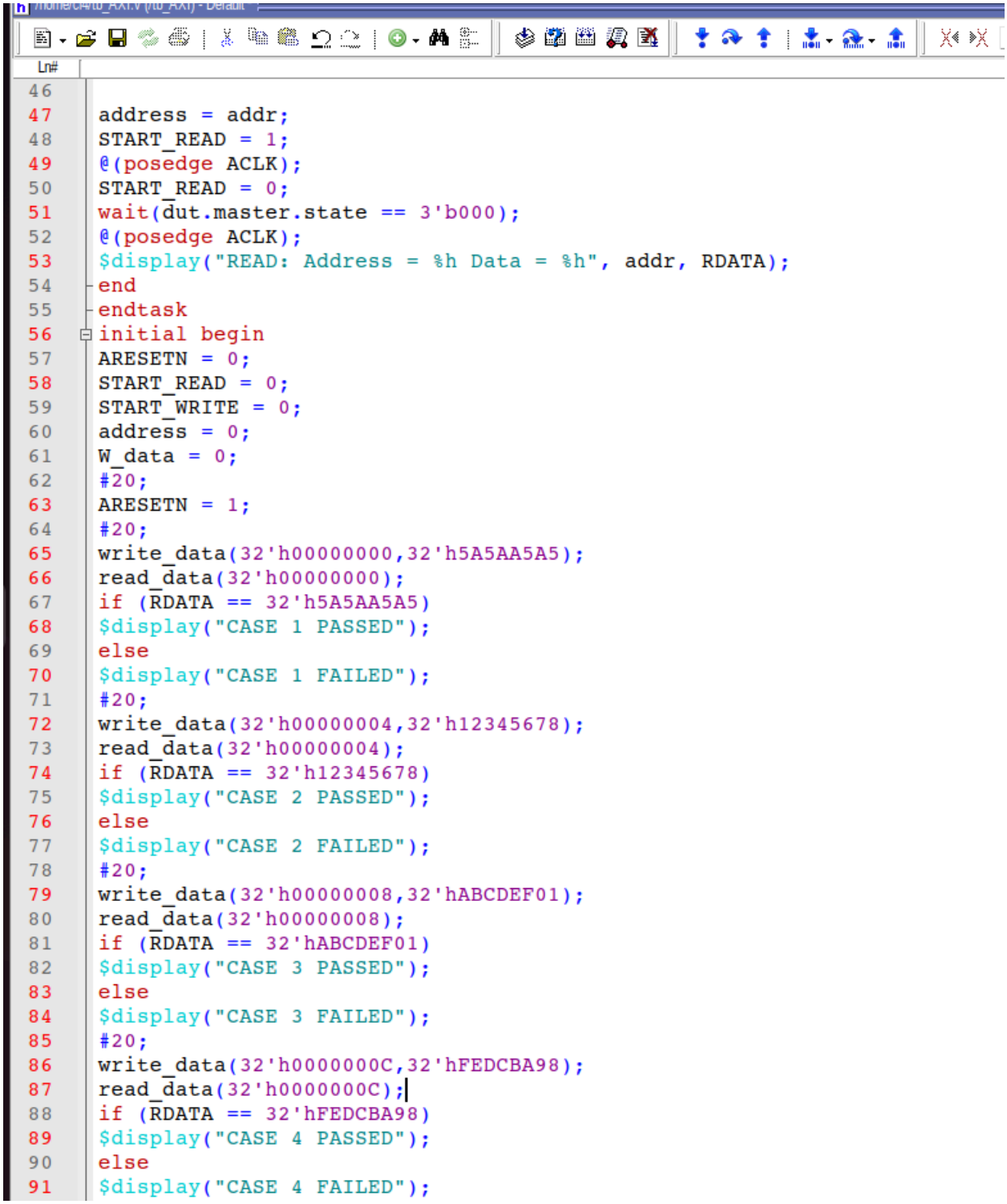
```
Ln#
 1  module AXI_top(
 2  input wire ACLK,
 3  input wire ARESETN,
 4  input wire START_READ,
 5  input wire START_WRITE,
 6  input wire [31:0] address,
 7  input wire [31:0] W_data,
 8  output wire [31:0] RDATA,
 9  output wire [1:0] RRESP,
10  output wire [1:0] BRESP
11  );
12  wire [31:0] ARADDR;
13  wire ARVALID;
14  wire ARREADY;
15  wire RREADY;
16  wire RVALID;
17  wire [31:0] AWADDR;
18  wire AWVALID;
19  wire AWREADY;
20  wire [31:0] WDATA;
21  wire [3:0] WSTRB;
22  wire WVALID;
23  wire WREADY;
24  wire BREADY;
25  wire BVALID;
26  AXI_master master(
27  .ACLK(ACLK),
28  .ARESETN(ARESETN),
29  .START_READ(START_READ),
30  .START_WRITE(START_WRITE),
31  .address(address),
32  .W_data(W_data),
33  .ARREADY(ARREADY),
34  .RDATA(RDATA),
35  .RRESP(RRESP),
36  .RVALID(RVALID),
37  .AWREADY(AWREADY),
38  .WREADY(WREADY),
39  .BRESP(BRESP),
40  .BVALID(BVALID),
41  .ARADDR(ARADDR),
42  .ARVALID(ARVALID),
43  .RREADY(RREADY),
44  .AWADDR(AWADDR),
45  .AWVALID(AWVALID),
46  .WDATA(WDATA),
```

```
44      .AWADDR(AWADDR),
45      .AWVALID(AWVALID),
46      .WDATA(WDATA),
47      .WSTRB(WSTRB),
48      .WVALID(WVALID),
49      .BREADY(BREADY)
50      );
51
52      AXI_slave slave(
53      .ACLK(ACLK),
54      .ARESETN(ARESETN),
55      .ARADDR(ARADDR),
56      .ARVALID(ARVALID),
57      .RREADY(RREADY),
58      .AWADDR(AWADDR),
59      .AWVALID(AWVALID),
60      .WDATA(WDATA),
61      .WSTRB(WSTRB),
62      .WVALID(WVALID),
63      .BREADY(BREADY),
64      .ARREADY(ARREADY),
65      .RDATA(RDATA),
66      .RRESP(RRESP),
67      .RVALID(RVALID),
68      .AWREADY(AWREADY),
69      .WREADY(WREADY),
70      .BRESP(BRESP),
71      .BVALID(BVALID)
72      );
73
74      endmodule
75
```

Test Bench:



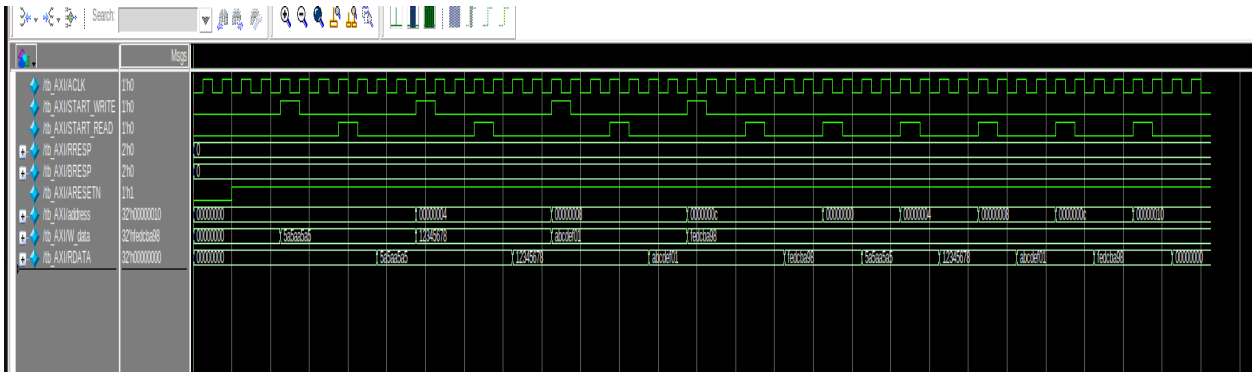
```
Ln#
 1  `timescale 1ns/1ps
 2  module tb_AXI;
 3  reg ACLK;
 4  reg ARESETN;
 5  reg START_READ;
 6  reg START_WRITE;
 7  reg [31:0] address;
 8  reg [31:0] W_data;
 9  wire [31:0] RDATA;
10  wire [1:0] RRESP;
11  wire [1:0] BRESP;
12  AXI_top dut(
13  .ACLK(ACLK),
14  .ARESETN(ARESETN),
15  .START_READ(START_READ),
16  .START_WRITE(START_WRITE),
17  .address(address),
18  .W_data(W_data),
19  .RDATA(RDATA),
20  .RRESP(RRESP),
21  .BRESP(BRESP)
22  );
23  initial begin
24  ACLK = 0;
25  forever #5 ACLK = ~ACLK;
26  end
27  task write_data;
28  input [31:0] addr;
29  input [31:0] data;
30  begin
31  @(posedge ACLK);
32  address = addr;
33  W_data = data;
34  START_WRITE = 1;
35  @(posedge ACLK);
36  START_WRITE = 0;
37  wait(dut.master.state == 3'b000);
38  @(posedge ACLK);
39  $display("WRITE: Address = %h Data = %h", addr, data);
40  end
41  endtask
42  task read_data;
43  input [31:0] addr;
44  begin
45  @(posedge ACLK);
46
```



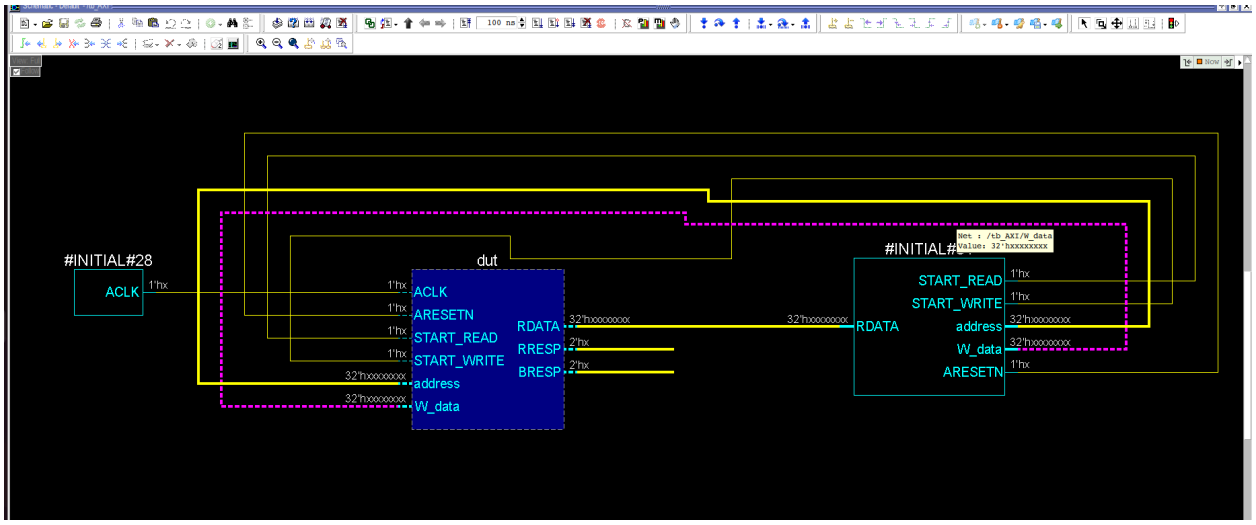
```
46
47  address = addr;
48  START_READ = 1;
49  @(posedge ACLK);
50  START_READ = 0;
51  wait(dut.master.state == 3'b000);
52  @(posedge ACLK);
53  $display("READ: Address = %h Data = %h", addr, RDATA);
54  end
55  endtask
56  initial begin
57  ARESETN = 0;
58  START_READ = 0;
59  START_WRITE = 0;
60  address = 0;
61  W_data = 0;
62  #20;
63  ARESETN = 1;
64  #20;
65  write_data(32'h00000000,32'h5A5AA5A5);
66  read_data(32'h00000000);
67  if (RDATA == 32'h5A5AA5A5)
68  $display("CASE 1 PASSED");
69  else
70  $display("CASE 1 FAILED");
71  #20;
72  write_data(32'h00000004,32'h12345678);
73  read_data(32'h00000004);
74  if (RDATA == 32'h12345678)
75  $display("CASE 2 PASSED");
76  else
77  $display("CASE 2 FAILED");
78  #20;
79  write_data(32'h00000008,32'hABCDEF01);
80  read_data(32'h00000008);
81  if (RDATA == 32'hABCDEF01)
82  $display("CASE 3 PASSED");
83  else
84  $display("CASE 3 FAILED");
85  #20;
86  write_data(32'h0000000C,32'hFEDCBA98);
87  read_data(32'h0000000C);
88  if (RDATA == 32'hFEDCBA98)
89  $display("CASE 4 PASSED");
90  else
91  $display("CASE 4 FAILED");
```

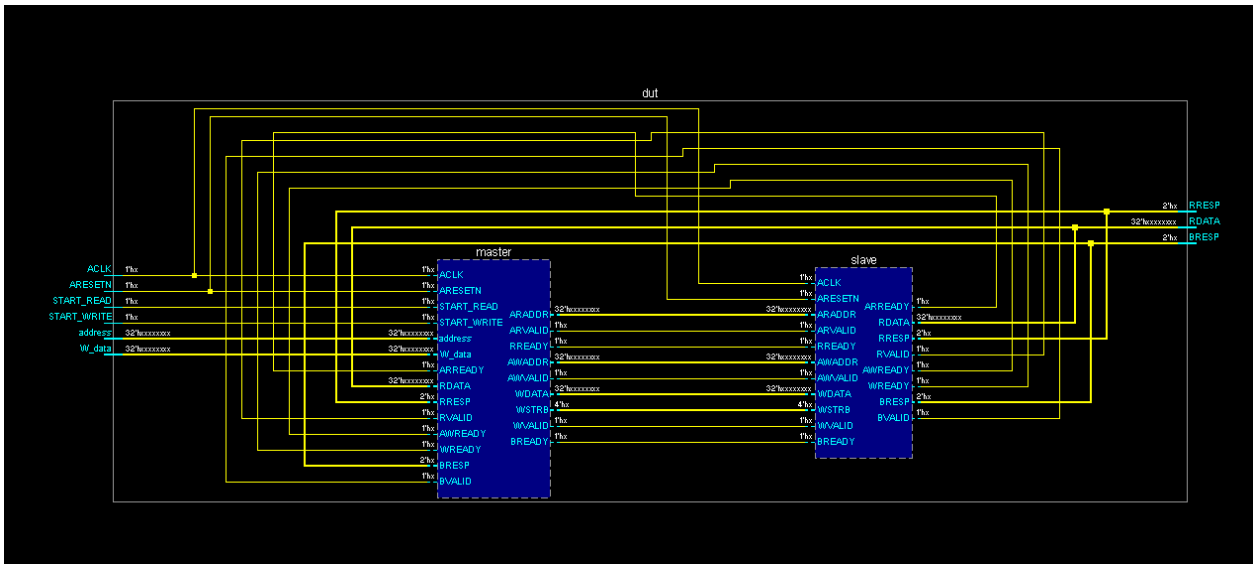
```
 85    #20;
 86    write_data(32'h0000000C,32'hFEDCBA98);
 87    read_data(32'h0000000C);
 88    if (RDATA == 32'hFEDCBA98)
 89    $display("CASE 4 PASSED");
 90    else
 91    $display("CASE 4 FAILED");
 92
 93    #20;
 94
 95    read_data(32'h00000000);
 96    if (RDATA == 32'h5A5AA5A5)
 97    $display("CASE 5 PASSED");
 98    else
 99    $display("CASE 5 FAILED");
100    #20;
101    read_data(32'h00000004);
102    if (RDATA == 32'h12345678)
103    $display("CASE 6 PASSED");
104    else
105    $display("CASE 6 FAILED");
106    #20;
107    read_data(32'h00000008);
108    if (RDATA == 32'hABCDEF01)
109    $display("CASE 7 PASSED");
110    else
111    $display("CASE 7 FAILED");
112    #20;
113    read_data(32'h0000000C);
114    if (RDATA == 32'hFEDCBA98)
115    $display("CASE 8 PASSED");
116    else
117    $display("CASE 8 FAILED");
118    #20;
119    read_data(32'h00000010);
120    if (RDATA == 32'h00000000)
121    $display("CASE 9 PASSED");
122    else
123    $display("CASE 9 FAILED");
124    #20;
125    $display("ALL TEST CASES COMPLETED");
126    $finish;
127    end
128    endmodule
129
130
```

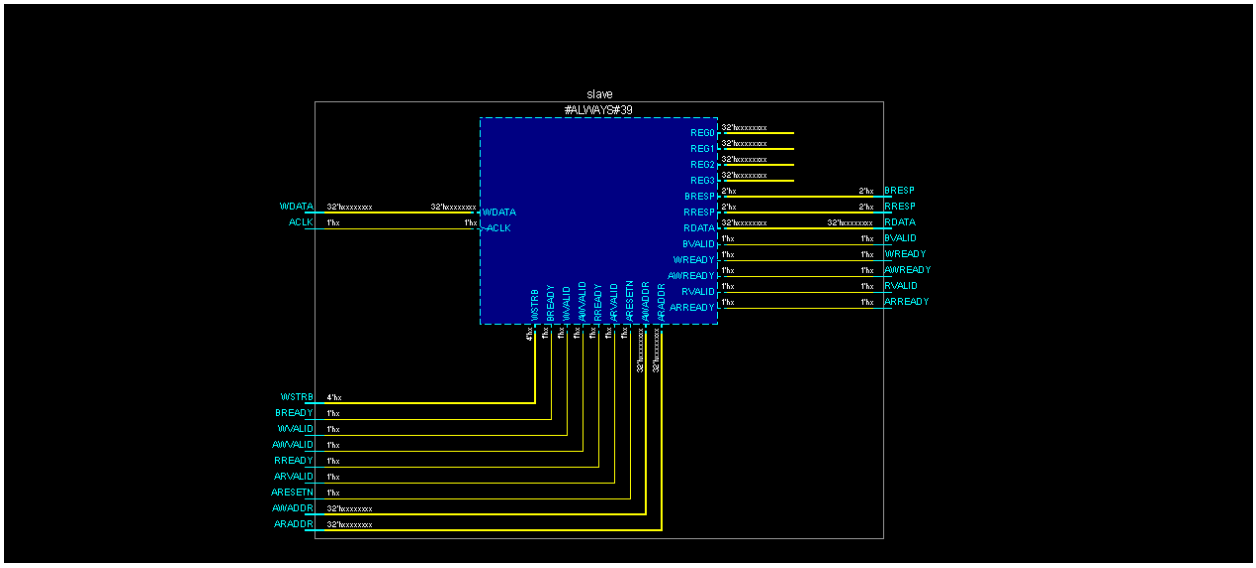
Simulation:

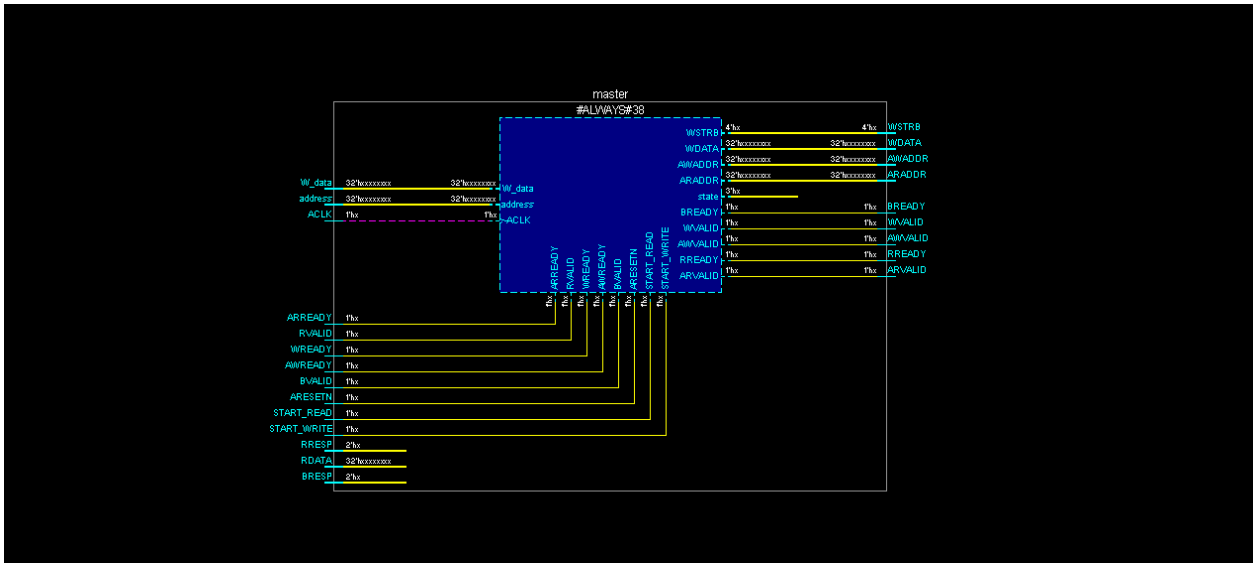

/tb_AXI/ACLK
/tb_AXI/START_WRITE
/tb_AXI/START_READ
/tb_AXI/RRESP
/tb_AXI/BRESP
/tb_AXI/ARESETN
/tb_AXI/address
/tb_AXI/W_data
/tb_AXI/RDATA

Schematic:


#INITIAL#28
ACLK
dut
ACLK
ARESETN
START_READ
START_WRITE
address
W_data
RDATA
RRESP
BRESP
#INITIAL#
START_READ
START_WRITE
address
W_data
ARESETN
RDATA


dut
master
slave
ACLK
ARESETN
START_READ
START_WRITE
address
W_data
RRESP
RDATA
BRESP


slave
#ALWAYS#39
WDATA
ACLK
WSTRB
BREADY
WVALID
AWVALID
RREADY
ARVALID
ARESETN
AWADDR
ARADDR
REG0
REG1
REG2
REG3
BRESP
RRESP
RDATA
BVALID
WREADY
AWREADY
RVALID
ARREADY


master
#ALWAYS#38
W_data
address
ACLK
WSTRB
WDATA
AWADDR
ARADDR
state
BREADY
WVALID
AWVALID
RREADY
ARVALID
ARREADY
RVALID
WREADY
AWREADY
BVALID
ARESETN
START_READ
START_WRITE
RRESP
RDATA
BRESP